

GAURAV TIWARI

SOFTWARE ENGINEER

+91 7054013976 | igauravtiwari1096@gmail.com | linkedin.com/in/gauravtiwarrii | github.com/gauravtiwarrii

SUMMARY

Final-year B.Tech CSE student, 2027 batch, who designs and ships full-stack products end to end: a 10-model Prisma/PostgreSQL group-expense platform live on Vercel, browser-side RSA-OAEP + AES-GCM message encryption over Socket.io, and a 5-node LangGraph research pipeline with a bounded reviewer critique cycle. Strongest in TypeScript and Next.js, with Python and SQL for data work. Seeking a full-time software engineering role.

TECHNICAL SKILLS

Languages: TypeScript, JavaScript, Python, SQL, Java, C++

Full-Stack: Next.js (App Router), React 19, Node.js, Express, Socket.io, REST APIs, Server-Sent Events, NextAuth v5, Zod, Tailwind CSS

Data & Persistence: PostgreSQL, MongoDB (Mongoose), Redis, Prisma, Supabase, Neon, SQLite, schema design and migrations

Python & ML: Pandas, NumPy, Scikit-Learn, XGBoost, Flask, Streamlit, BeautifulSoup

Tooling: Git, Docker and Docker Compose, Linux, Vercel, Render, WebCrypto API, LangGraph.js, Postman

Foundations: Data Structures and Algorithms, OOP, DBMS, Operating Systems, Computer Networks, Distributed Systems

PROJECTS

SplitSmart — Group Expense & Settlement Platform | [GitHub](#) | [Live Demo](#)

Jun 2026

Next.js 16, React 19, TypeScript, Prisma 7, PostgreSQL (Neon), NextAuth v5, Tailwind CSS

- Designed a 10-model PostgreSQL schema whose central invariant is a per-member active/inactive timeline — a member is liable only for expenses logged while active — which all four split modes (equal, exact amounts, percentages, custom share ratios) resolve against.
- Reduced the number of settlement transfers with a greedy debt-simplification pass that recursively matches the largest debtor against the largest creditor instead of settling every pairwise IOU.
- Gated CSV import behind a 5-step wizard running 12 anomaly rules before any write: duplicate rows, split totals that do not reconcile to the expense amount, transactions falling outside a member's active window, and Levenshtein fuzzy matching on misspelled member names.
- Made every mutation reconstructible with an append-only audit log capturing CREATE/UPDATE/DELETE/SETTLE/IMPORT events alongside JSON old-value vs new-value diffs, and enforced route protection in Next.js Edge middleware.

ChatPulse — End-to-End Encrypted Real-Time Chat | [GitHub](#)

Mar 2026

Node.js, Express, Socket.io, MongoDB (Mongoose), WebCrypto API, IndexedDB

- Implemented hybrid end-to-end encryption with the browser WebCrypto API: an RSA-OAEP keypair generated client-side with the private key held only in IndexedDB, each message encrypted under a fresh 256-bit AES-GCM key, and the payload carrying ciphertext plus that session key wrapped under the recipient's public key.
- Chose per-message AES-GCM over encrypting directly under RSA because RSA-OAEP caps plaintext at the modulus size and carries no authentication tag, so the key-wrap keeps message length unbounded and integrity-checked.
- Kept the server zero-knowledge: Socket.io relays and MongoDB persists ciphertext only, with no server-side code path able to decrypt stored messages.
- Expired one-time passcodes and ephemeral presence documents with Mongoose TTL indexes at the database level rather than an application-side cron sweep.

InvestIQ — Multi-Stage Research Pipeline | [GitHub](#) | [Live Demo](#)

Jun 2026

Next.js 15, TypeScript, LangGraph.js, Google Gemini 2.0 Flash, Tavily, Supabase, Server-Sent Events

- Built a 5-node LangGraph.js state graph over live Tavily search and Yahoo Finance data, with a fan-out stage issuing 5 analysis calls concurrently off a single shared research fetch.
- Implemented a conditional reject edge routing an unapproved thesis from the reviewer node back to the investment node, bounded so a disputed run terminates instead of looping indefinitely.
- Streamed stage-by-stage progress to the browser over Server-Sent Events instead of client-side polling, and persisted per-user report history to Supabase Postgres.

Distributed Web Crawler | [GitHub](#)

May 2026

Python, Redis, MongoDB, BeautifulSoup, Docker Compose, Streamlit

- Coordinated 5 worker threads through a Redis sorted-set frontier that ranks seed URLs above links discovered mid-crawl, plus a Redis visited-set index marked at dequeue time so concurrent workers never crawl the same page twice.
- Enforced politeness with a per-domain robots.txt parser cached on first fetch and a 1.0-second minimum gap between requests to the same domain, capping throughput by design rather than by worker count.
- Parsed titles, meta descriptions, readable text and links into MongoDB, shipping Redis and MongoDB as Docker Compose services for one-command bring-up with queue depth and pages-crawled on a Streamlit dashboard.

CERTIFICATIONS

• **Oracle Cloud Infrastructure 2025 Certified Data Science Professional** — Oracle

Aug 2025

• **Cloud Computing** — NPTEL

Nov 2025

• **CodeQuest DSA Summer Bootcamp** — 100+ problems across arrays, graphs and dynamic programming

Jun 2025

EDUCATION

Lovely Professional University, Phagwara, Punjab

Aug 2023 - May 2027

B.Tech, Computer Science and Engineering | CGPA 7.26

Lions School, Mirzapur, Uttar Pradesh

Mar 2019 - May 2022

Intermediate (PCM), 74.6%